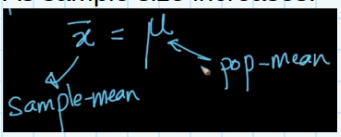
Probability and Statistics

Random Variable:
A random variable (denoted by X) is a variable whose possible values are the outcomes of an experiment.
E.g. tossing a coin:
Random variable Y = { H, T }
P(Y=H) = ½ (assuming the coin is a fair one)

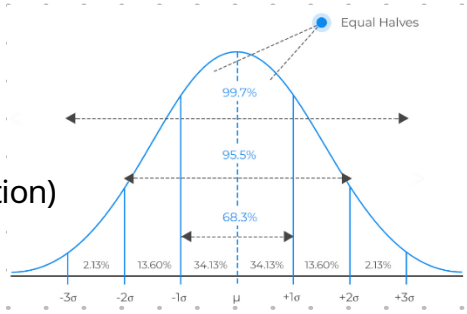
P(X=x) = P(x) = Probability that random variable X takes the value x.

Population: set of similar items or events which is of interest for some experiment
Sample: subset of population chosen to represent it in an analysis

As sample size increases:



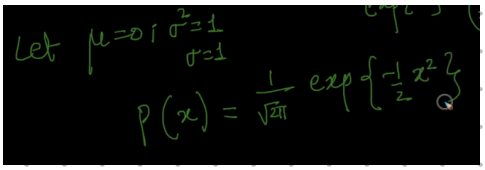
Gaussian Distribution (Normal) $\mathcal{N}(\mu, \sigma^2)$



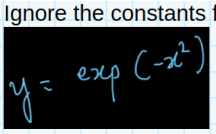
μ: mean, σ^2= variance (parameters of the normal distribution)

PDF: $$\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

Let's make some assumptions to simplify the PDF and understand the shape.

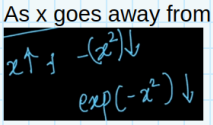


Ignore the constants for now,

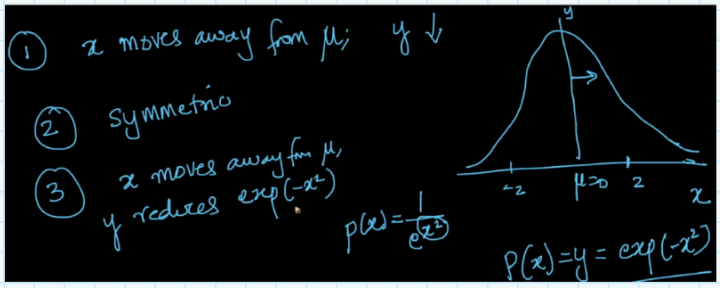


This gives a bell curve.

As x goes away from zero,

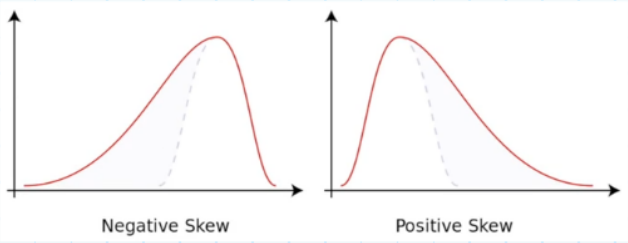


Conclusions:



Skewness: asymmetry

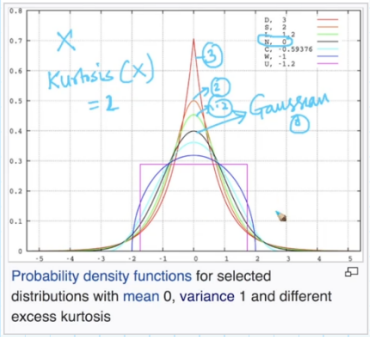
Negative skew: Left tail is longer
Positive skew: Right tail is longer



Kurtosis: tailed-ness
Kurtosis describes the "fatness" of the tails found in probability distributions.

There are three kurtosis categories—
mesokurtic (normal),
platykurtic (less than normal),
and leptokurtic (more than normal).

Kurtosis is a measure of the combined weight of a distribution's tails relative to the center of the distribution curve (the mean).

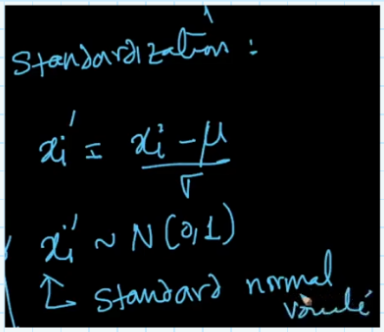


Probability density functions for selected distributions with mean 0, variance 1 and different excess kurtosis

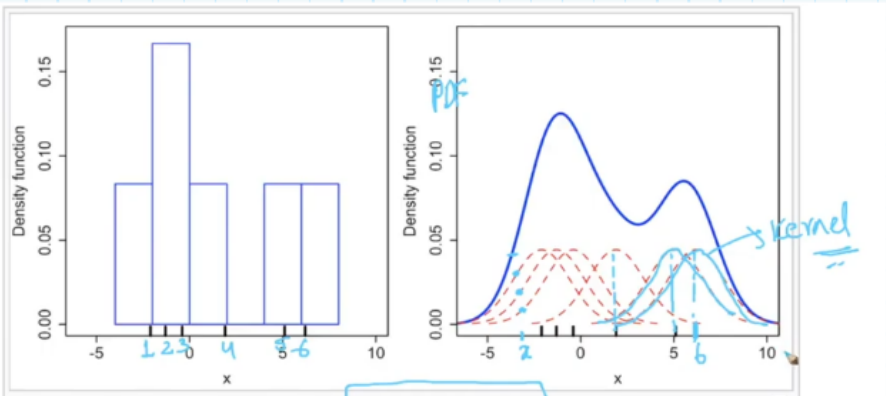
Standard Normal Distribution

Z ~ N(0, 1)

We can convert a normal distribution to standard normal

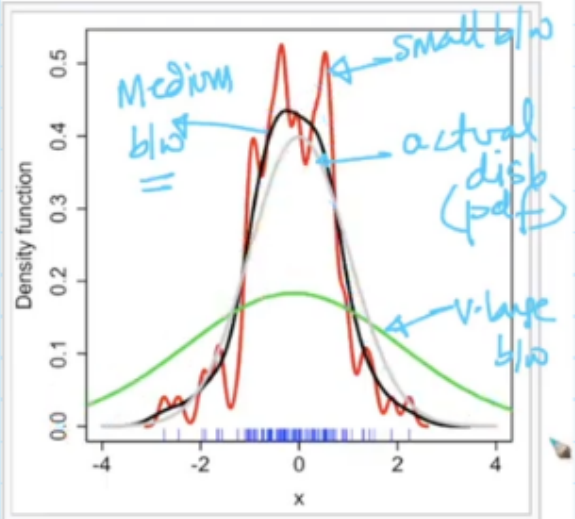


Kernel Density Estimation



Comparison of the histogram (left) and kernel density estimate (right) constructed using the same data. The 6 individual kernels are the red dashed curves, the kernel density estimate the blue curves. The data points are the rug plot on the horizontal axis.

We have a number of observations. We draw individual Gaussian kernels for each.
For a kernel, the mean is the observation. The variance is called the bandwidth.
For any x, the corresponding red curve points (kernel) are added up to get the corresponding kernel density (blue).

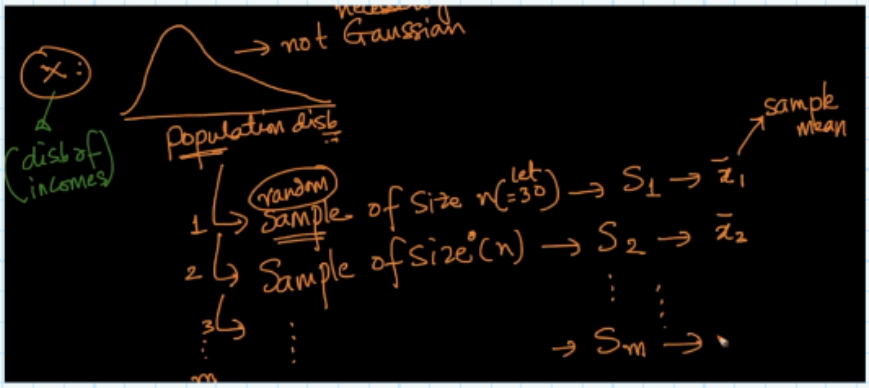


Kernel density estimate (KDE) with different bandwidths of a random sample of 100 points from a standard normal distribution. Grey: true density (standard normal). Red: KDE with h=0.05. Black: KDE with h=0.337. Green: KDE with h=2.

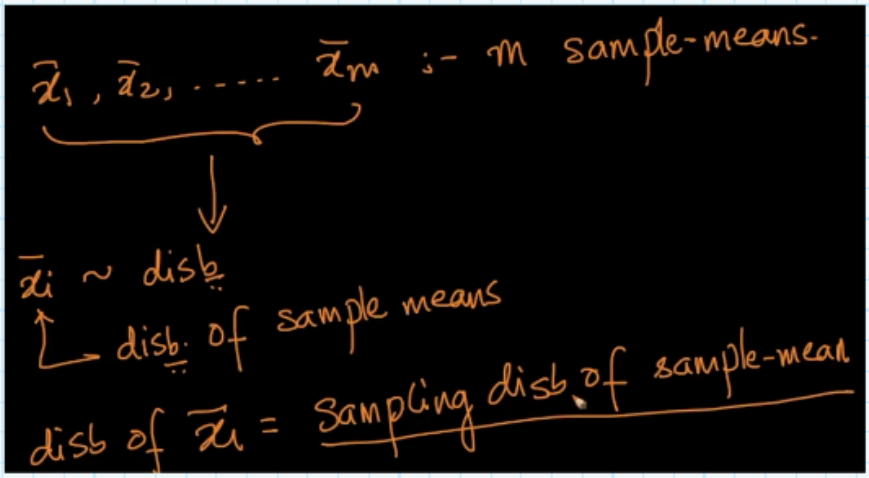
An optimal bandwidth would be a good estimate of the actual distribution.

Sampling Distribution
a probability distribution of a statistic obtained from a larger number of samples drawn from a specific population.

The standard deviation of the sampling distribution of a statistic is referred to as the standard error of that quantity.



Say we sample multiple times. We get sample sets S1, S2, …, Sm
Each of those has some mean.
Those sample means also have a distribution. Their distributions is called the sample distribution of sample means.



Central Limit Theorem

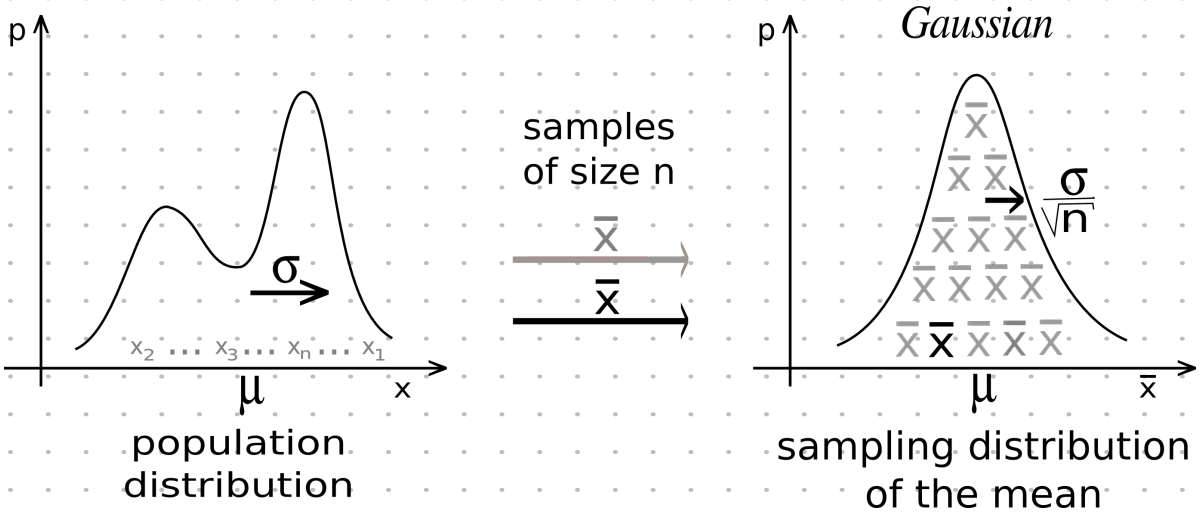
X is a random variable with finite mean and variance.
We sample. S1, S2, ..., Sm (samples of size n)
We can compute sample mean for each.

$\bar{x_1}, \bar{x_2}, \bar{x_3}, ..., \bar{x_m}$

Distribution of $\bar{x_i}$ is the sampling distribution of sample means.

CLT says that x_i is distributed with a Gaussian distribution $\mathcal{N}(\mu, \frac{\sigma^2}{n})_{n \to \infty}$

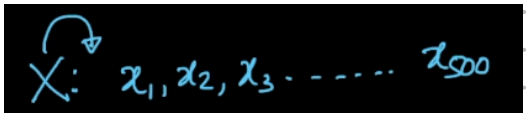
the central limit theorem (CLT) states that, under appropriate conditions,
the distribution of a normalized version of the sample mean converges to a standard normal distribution.
This holds even if the original variables themselves are not normally distributed.



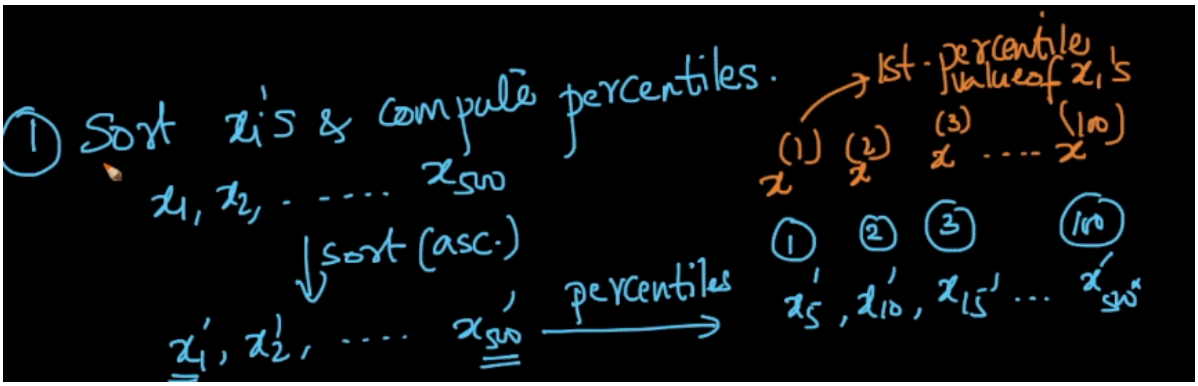
As a general rule, sample sizes of around 30-50 are deemed sufficient for the CLT to hold, meaning that the distribution of the sample means is fairly normally distributed.

Quantile-Quantile (QQ) Plot

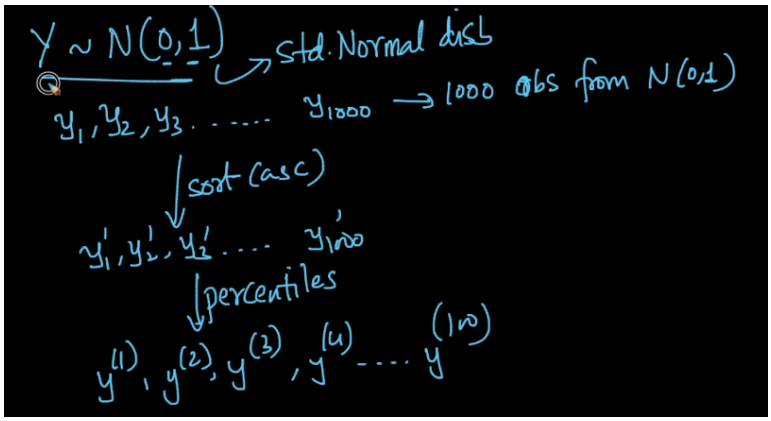
We have a random variable X and 500 observations.
QQ plots help us determine if X is Gaussian.



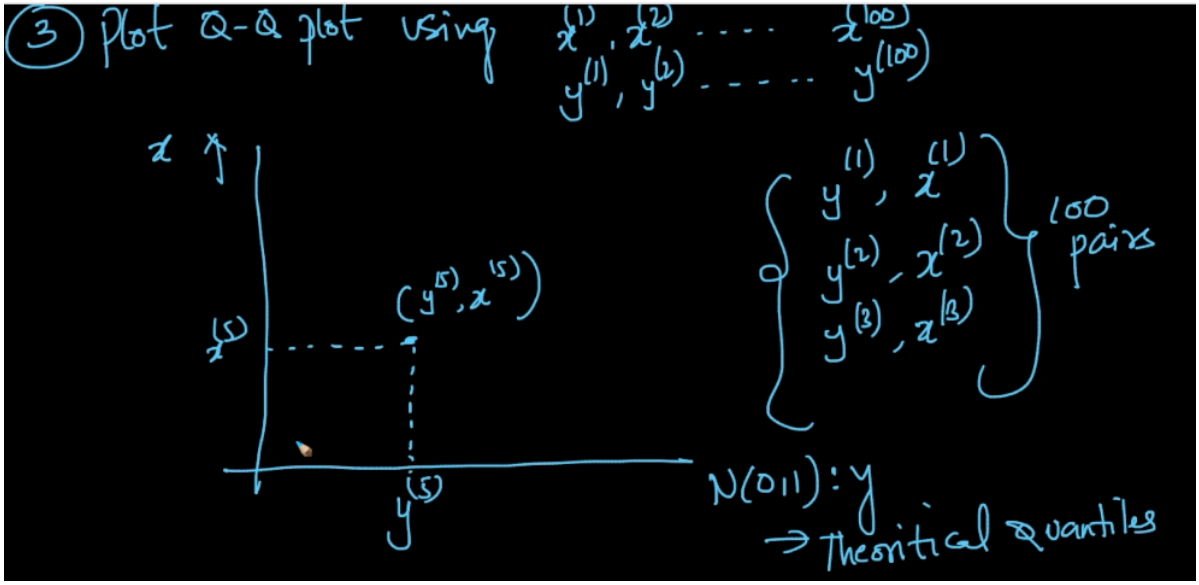
Steps:
1. We sort our xi's



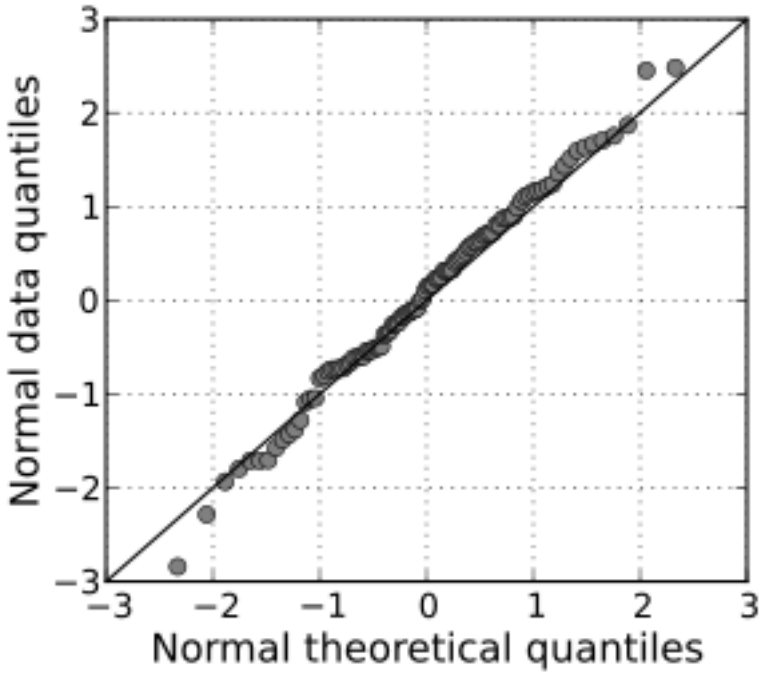
2. Consider a rv Y (std normal)
Get observations from distribution.
Sort. Compute percentiles.

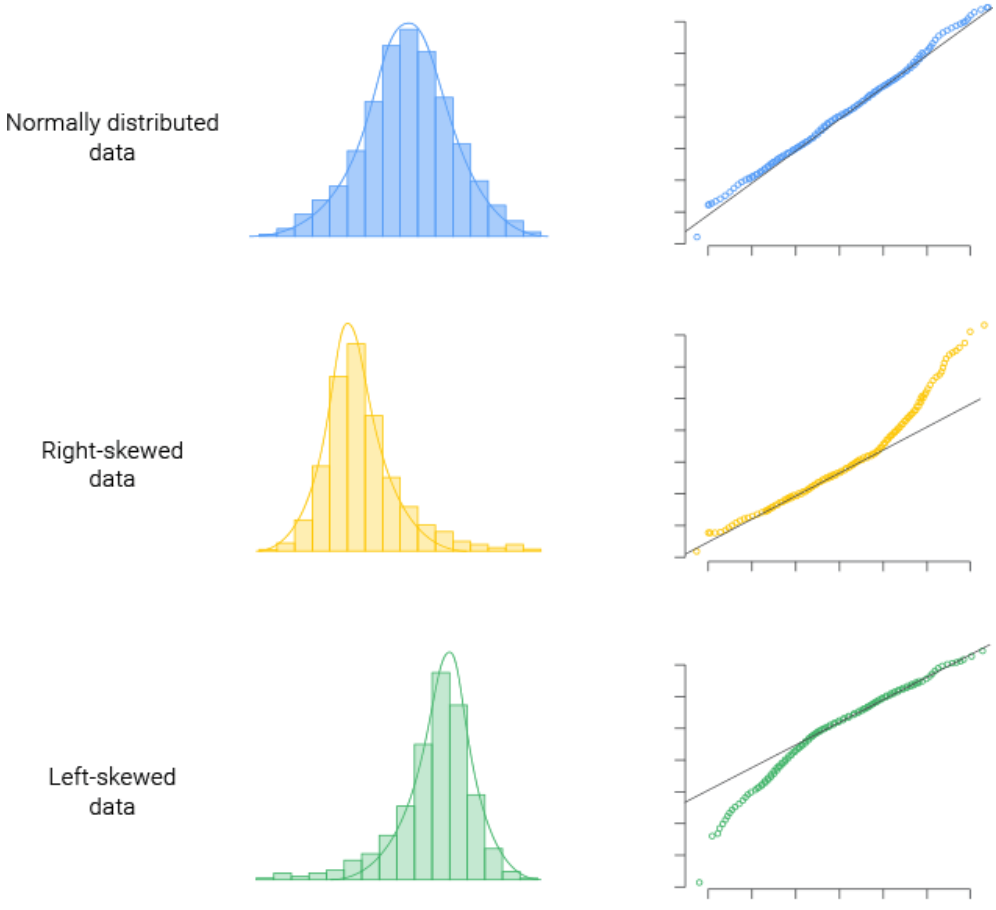


3. Plot QQ



A QQ plot looks like:





QQ plots tell us if 2 random variables have the same distribution

Q-Q Plot

```
#Q-Q plot
import numpy as np
import pylab
import scipy.stats as stats

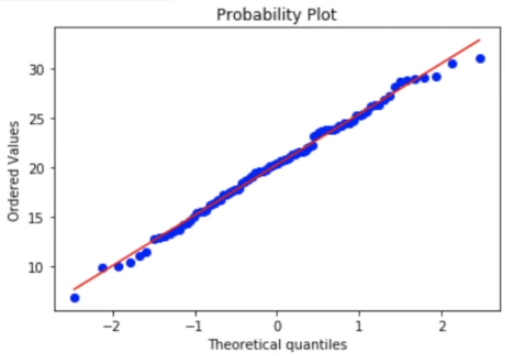
# N(0,1)
std_normal = np.random.normal(loc = 0, scale = 1, size=1000)

# 0 to 100th percentiles of std-normal
for i in range(0,101):
    print(i, np.percentile(std_normal,i))
```

```
0 -3.26635243798
1 -2.42524743724
2 -2.10830577037
3 -1.89195040895
4 -1.75583511997
5 -1.64000299011
6 -1.54972268746
7 -1.44604694292
8 -1.34081881034
9 -1.30146314071
10 -1.26285915899
11 -1.19258025826
```

```
# generate 100 sanples from N(20,5)
measurements = np.random.normal(loc = 20, scale = 5, size=100)
#try size=1000

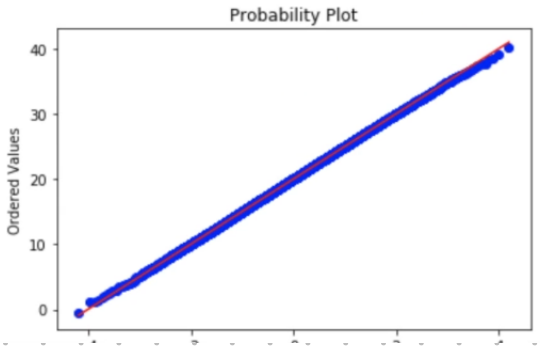
stats.probplot(measurements, dist="norm", plot=pylab)
pylab.show()
```



say we increase size from 100 to 50,000. The dist becomes more normal

```
measurements = np.random.normal(loc = 20, scale = 5, size=50000)
#try size=1000

stats.probplot(measurements, dist="norm", plot=pylab)
pylab.show()
```



QQ plots don't work well with a smaller # of observations

Chebyshev's Inequality

If we have X that is normally distributed, we have the 68-95-99.7 rule.
Say X is the heights of students. If we know that X follows the Gaussian distribution, and the mean and variance, then

$$P(\mu - 2\sigma \leq x \leq \mu + 2\sigma) = 95\%$$

if mean is 150 cm and std dev is 10 cm then 95% students lie between [130, 170]

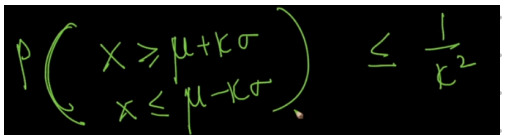
What if we just know the mean (is finite) and std dev (is non zero and finite), but not what the distribution is?
Can we say that x% of data lies between 2 std dev and all that?

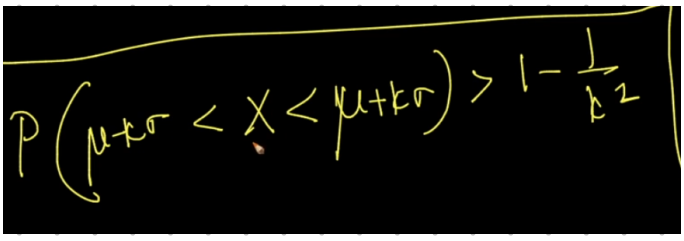
Let X be a rand var with finite non zero variance σ^2 (and thus finite expected value μ)
For any real k > 0

$$\Pr(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}.$$

Only the case k > 1 k > 1 is useful.

same as saying



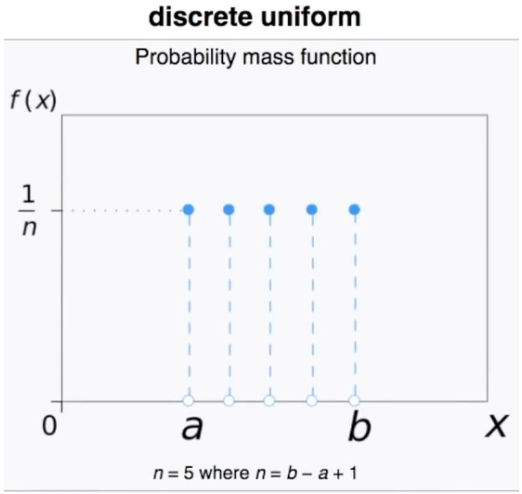


Uniform Distribution

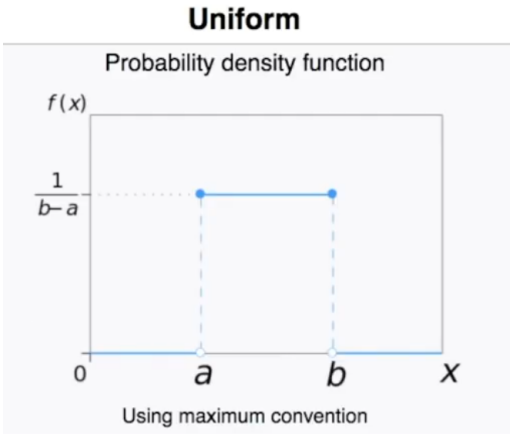
finite # of values are equally likely to be observed

ex. throwing a dice

every one of n observations has a prob 1/n



continuous



Notation	$\mathcal{U}(a,b)$ or $\operatorname{unif}(a,b)$
Parameters	$-\infty < a < b < \infty$
Support	$x \in [a,b]$
PDF	$\begin{cases} \frac{1}{b-a} & \text{for } x \in [a,b] \\ 0 & \text{otherwise} \end{cases}$
CDF	$\begin{cases} 0 & \text{for } x < a \\ \frac{x-a}{b-a} & \text{for } x \in [a,b) \\ 1 & \text{for } x \geq b \end{cases}$
Mean	$\frac{1}{2}(a+b)$
Median	$\frac{1}{2}(a+b)$
Mode	any value in (a,b)
Variance	$\frac{1}{12}(b-a)^2$
Skewness	0

Bernoulli Distribution

takes 2 values
  1 with prob p
  0 with prob 1-p

Parameters	$0 < p < 1, p \in \mathbb{R}$
Support	$k \in \{0, 1\}$
pmf	$\begin{cases} q = (1-p) & \text{for } k = 0 \\ p & \text{for } k = 1 \end{cases}$
CDF	$\begin{cases} 0 & \text{for } k < 0 \\ 1-p & \text{for } 0 \le k < 1 \\ 1 & \text{for } k \ge 1 \end{cases}$
Mean	p
Median	$\begin{cases} 0 & \text{if } q > p \\ 0.5 & \text{if } q = p \\ 1 & \text{if } q < p \end{cases}$
Mode	$\begin{cases} 0 & \text{if } q > p \\ 0, 1 & \text{if } q = p \\ 1 & \text{if } q < p \end{cases}$
Variance	$p(1-p) (= pq)$
Skewness	$1 - 2p$

Binomial Distribution

  parameters n and p

  discrete prob dist of # of successes in n independent trials,
  each asking a yes-no question,
  each with its own boolean valued outcome (0/1)

  single trial is Bernoulli trial

Notation	$B(n, p)$
Parameters	$n \in \mathbb{N}_0$ — number of trials $p \in [0,1]$ — success probability in each trial
Support	$k \in \{0, \ldots, n\}$ — number of successes
pmf	$\binom{n}{k} p^k (1-p)^{n-k}$
CDF	$I_{1-p}(n-k, 1+k)$
Mean	np
Median	$\lfloor np \rfloor$ or $\lceil np \rceil$
Mode	$\lfloor (n+1)p \rfloor$ or $\lceil (n+1)p \rceil - 1$
Variance	$np(1-p)$
Skewness	$\frac{1-2p}{\sqrt{np(1-p)}}$
Ex. kurtosis	$\frac{1-6p(1-p)}{np(1-p)}$

Log Normal Distribution

distribution of random variable
whose logarithm is normally distributed

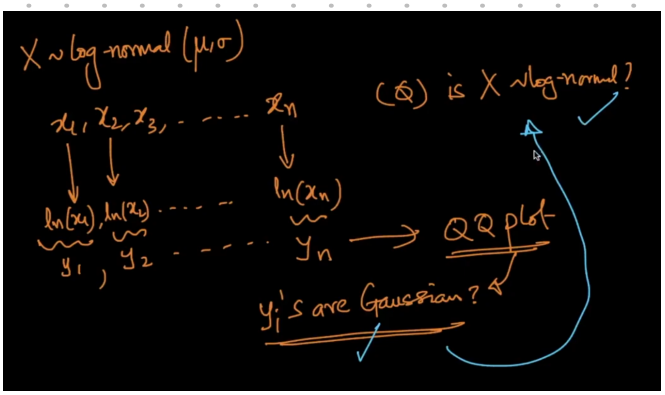
If X is log-normally distributed,
then Y = ln(X) is normally distributed.

Mean	$\exp(\mu + \sigma^2/2)$
Median	$\exp(\mu)$
Mode	$\exp(\mu - \sigma^2)$
Variance	$[\exp(\sigma^2) - 1]\exp(2\mu + \sigma^2)$
Skewness	$(e^{\sigma^2}+2)\sqrt{e^{\sigma^2}-1}$
Ex. kurtosis	$\exp(4\sigma^2) + 2\exp(3\sigma^2) + 3\exp(2\sigma^2) - 6$
Entropy	$\log(\sigma e^{\mu+\frac{1}{2}}\sqrt{2\pi})$

Examples include the following:

- Human behaviors
  - The length of comments posted in Internet discussion forums follows a log-normal distribution.[18]
  - The users' dwell time on the online articles (jokes, news etc.) follows a log-normal distribution.[19]
- In biology and medicine,
  - Measures of size of living tissue (length, skin area, weight);[20]
  - For highly communicable epidemics, such as SARS in 2003, if publication intervention is involved, the number of hospitalized cases is shown to satisfy the lognormal distribution with no free parameters if an entropy is assumed and the standard deviation is determined by the principle of maximum rate of entropy production.[21]

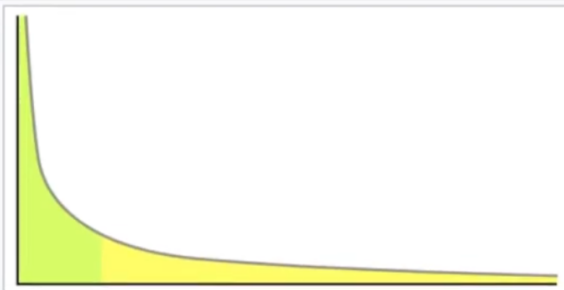
To check if X is log-normally distributed?



Power Law

functional relationship between 2 quantities,
relative change in one results in proportional relative change in the other

e.g. consider a square. length becomes twice -> area becomes 4 times

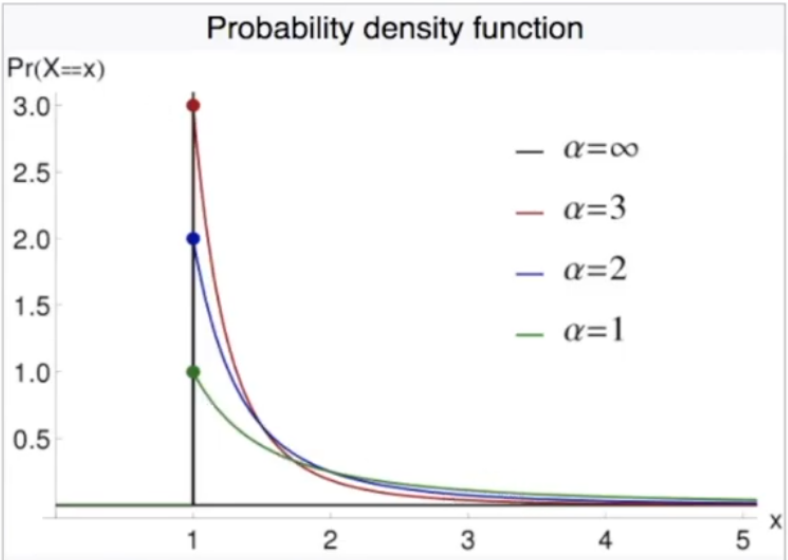


An example power-law graph, being used to demonstrate ranking of popularity. To the right is the long tail, and to the left are the few that dominate (also known as the 80–20 rule).

Pareto Distribution
follows power law

xm is like the mu in Gaussian
shape is like the sigma

Pareto Type I



Pareto Type I probability density functions for various α with $x_m = 1$. As $\alpha \to \infty$ the distribution approaches $\delta(x - x_m)$ where δ is the Dirac delta function.

Parameters	$x_m > 0$ scale (real) $\alpha > 0$ shape (real)
Support	$x \in [x_m, +\infty)$
PDF	$\frac{\alpha x_m^\alpha}{x^{\alpha+1}}$ for $x \geq x_m$
CDF	$1 - \left(\frac{x_m}{x}\right)^\alpha$ for $x \geq x_m$
Mean	$\begin{cases} \infty & \text{for } \alpha \leq 1 \\ \frac{\alpha x_m}{\alpha - 1} & \text{for } \alpha > 1 \end{cases}$
Median	$x_m \sqrt[\alpha]{2}$

here as α reduces,
the tail becomes fatter

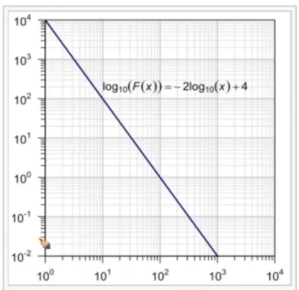
when $\alpha = \infty$
the pdf looks like a delta function
(a function with one peak value,
rest values are exactly zero)

To check if something follows
the Pareto distribution: log-log plot
instead of plotting (x, y), plot (logx, logy)
if straight line, then yes

You can also use QQ plots:
Check if x & y follow same distribution
by keeping Y a Pareto distribution

There's another: Hill-horror plot

Log–log plots are an alternative way of graphically examining the tail of a distribution using a random sample. This method consists of plotting the logarithm of an estimator of the probability that a particular number of the distribution occurs versus the logarithm of that particular number. Usually, this estimator is the proportion of times that the number occurs in the data set. If the points in the plot tend to "converge" to a straight line for large numbers in the x axis, then the researcher concludes that the distribution has a power-law tail. Examples of the application of these types of plot have



A straight line on a log–log plot is strong evidence for power-laws, the slope of the straight line corresponds to the power law exponent.

Power Transform (Box-Cox Transform)

we can convert log normal to gaussian by taking $\ln x_i$ of each value
we convert power law distributed using Box-Cox Transform

$$box_cox(x_i) = \lambda$$

$$y_i = \frac{x_i^\lambda - 1}{\lambda} if \lambda \neq 0$$

$$= lg(x_i) if \lambda = 0$$

as if $\lambda = 0$ then X is log-normal

scipy.stats.boxcox

scipy.stats.boxcox(*x, lmbda=None, alpha=None, optimizer=None*) [source]

Return a dataset transformed by a Box-Cox power transformation.

Parameters: **x** : *ndarray*

Input array to be transformed.

If *lmbda* is not None, this is an alias of `scipy.special.boxcox`. Returns nan if `x < 0`; returns -inf if `x == 0 and lmbda < 0`.

If *lmbda* is None, array must be positive, 1-dimensional, and non-constant.

lmbda : *scalar, optional*

If *lmbda* is None (default), find the value of *lmbda* that maximizes the log-likelihood function and return it as the second output argument.

If *lmbda* is not None, do the transformation for that value.

alpha : *float, optional*

If *lmbda* is None and `alpha` is not None (default), return the `100 * (1-alpha)%` confidence interval for *lmbda* as the third output argument. Must be between 0.0 and 1.0.

If *lmbda* is not None, `alpha` is ignored.

optimizer : *callable, optional*

If *lmbda* is None, *optimizer* is the scalar optimizer used to find the value of *lmbda* that minimizes the negative log-likelihood function. *optimizer* is a callable that accepts one argument:

fun : *callable*

The objective function, which evaluates the negative log-likelihood function at a provided value of *lmbda*

and returns an object, such as an instance of `scipy.optimize.OptimizeResult`, which holds the optimal value of *lmbda* in an attribute *x*.

See the example in `boxcox_normmax` or the documentation of `scipy.optimize.minimize_scalar` for more information.

If *lmbda* is not None, *optimizer* is ignored.

Examples

```
>>> from scipy import stats
>>> import matplotlib.pyplot as plt
```

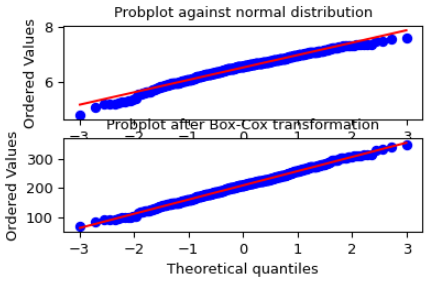
We generate some random variates from a non-normal distribution and make a probability plot for it, to show it is non-normal in the tails:

```
>>> fig = plt.figure()
>>> ax1 = fig.add_subplot(211)
>>> x = stats.loggamma.rvs(5, size=500) + 5
>>> prob = stats.probplot(x, dist=stats.norm, plot=ax1)
>>> ax1.set_xlabel('')
>>> ax1.set_title('Probplot against normal distribution')
```

We now use `boxcox` to transform the data so it's closest to normal:

```
>>> ax2 = fig.add_subplot(212)
>>> xt, _ = stats.boxcox(x)
>>> prob = stats.probplot(xt, dist=stats.norm, plot=ax2)
>>> ax2.set_title('Probplot after Box-Cox transformation')
```

```
>>> plt.show()
```



Covariance: Covariance measures the total variation of two random variables from their expected values.

$$\operatorname{cov}(X,Y) = \operatorname{E}\left[(X - \operatorname{E}[X])(Y - \operatorname{E}[Y])\right]$$

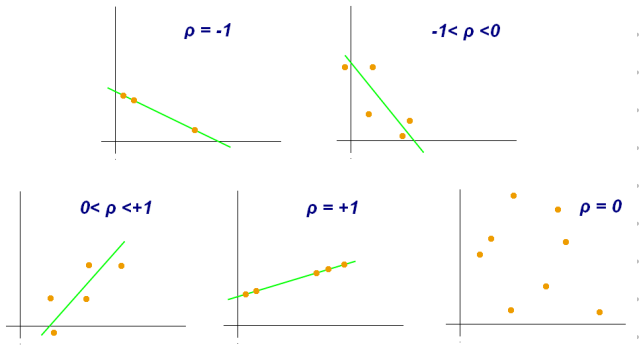
$$cov_{x,y} = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{N-1}$$

N if pop mean is known
N-1 for sample mean

If as X increases, Y also increases, then cov(X, Y) = +ve
If as X increases, Y decreases, then cov(X, Y) = -ve

Change in the metric system may lead to incorrect conclusions from the cov

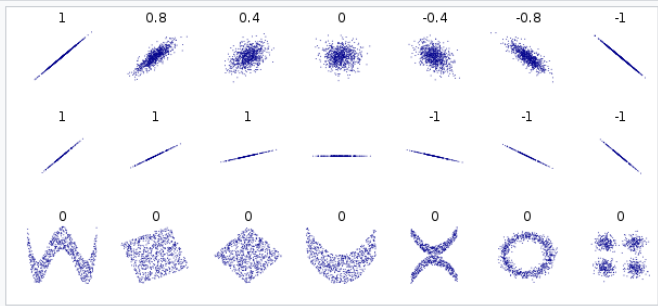
Pearson Correlation Coefficient $\rho_{X,Y} = \frac{\operatorname{cov}(X,Y)}{\sigma_X \sigma_Y}$



$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

- tells the linear correlation betn 2 sets of data.
- it is like a normalized version of covariance.
- lies betn -1 and +1

Both covariance and PCC are biased towards linear relationships



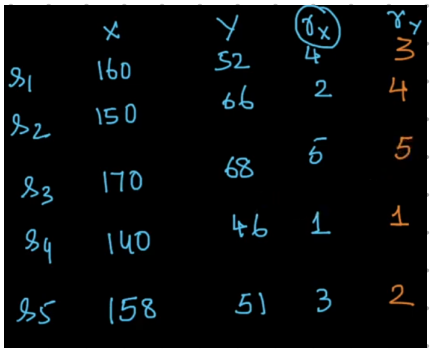
Several sets of (*x*, *y*) points, with the correlation coefficient of *x* and *y* for each set. The correlation reflects the strength and direction of a linear relationship (top row), but not the slope of that relationship (middle), nor many aspects of nonlinear relationships (bottom). N.B.: the figure in the center has a slope of 0 but in that case the correlation coefficient is undefined because the variance of *Y* is zero.

values of PCC may not give complete information for non-linear relationships

Spearman's Rank Correlation Coefficient

$$r_s = \rho_{\mathrm{R}(X),\mathrm{R}(Y)} = \frac{\mathrm{cov}(\mathrm{R}(X),\mathrm{R}(Y))}{\sigma_{\mathrm{R}(X)}\sigma_{\mathrm{R}(Y)}}$$

"It assesses how well the relationship between two variables can be described using a monotonic function."



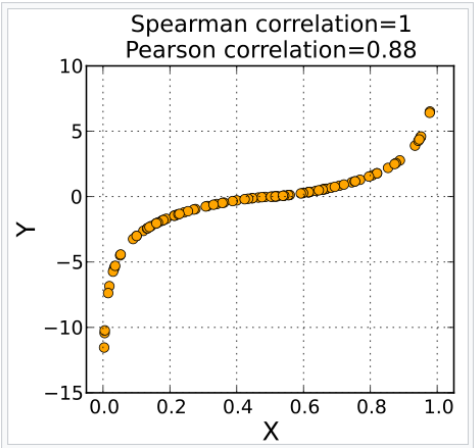
ranks are just ascending orderings for r_x,
140 is smallest so it gets rank 1, then 150 gets 2 and so on

now we just calculate PCC for the ranks

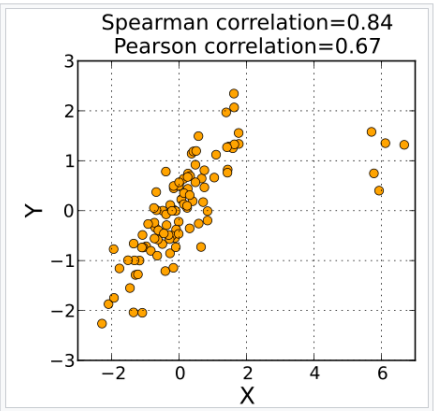
Spearman's is 1
if
{
  if x2 > x1 then y2 > y1
}
holds true for any two points

it doesn't care about how far things are, how close they are, and so on
it just cares about the monotonicity of rank

also it doesn't care about outliers



A Spearman correlation of 1 results when the two variables being compared are monotonically related, even if their relationship is not linear. This means that all data points with greater x values than that of a given data point will have greater y values as well. In contrast, this does not give a perfect Pearson correlation.



The Spearman correlation is less sensitive than the Pearson correlation to strong outliers that are in the tails of both samples. That is because Spearman's ρ limits the outlier to the value of its rank.

less sensitive to outliers

Correlation does not imply Causation

```
This fallacy is
also known by the Latin phrase
cum hoc ergo propter hoc
('with this, therefore because of this').
```

Correlation means there is a relationship or pattern between the values of two variables. A scatterplot displays data about two variables as a set of points in the xy-plane and is a useful tool for determining if there is a correlation between the variables.

Causation means that one event causes another event to occur. Causation can only be determined from an appropriately designed experiment. In such experiments, similar groups receive different treatments, and the outcomes of each group are studied. We can only conclude that a treatment *causes* an effect if the groups have noticeably different outcomes.



Dinosaur illiteracy and extinction may be correlated, but that would not mean the variables had a causal relationship.

Dinosaur illiteracy and extinction may be correlated, but that would not mean the variables had a causal relationship.

B causes A (reverse causation or reverse causality) [edit]

Reverse causation or **reverse causality** or **wrong direction** is an informal fallacy of questionable cause where cause and effect are reversed. The cause is said to be the effect and vice versa.

Example 1

The faster that windmills are observed to rotate, the more wind is observed.

Therefore, wind is caused by the rotation of windmills. (Or, simply put: windmills, as their name indicates, are machines used to produce wind.)

In this example, the correlation (simultaneity) between windmill activity and wind velocity does not imply that wind is caused by windmills. It is rather the other way around, as suggested by the fact that wind does not need windmills to exist, while windmills need wind to rotate. Wind can be observed in places where there are no windmills or non-rotating windmills—and there are good reasons to believe that wind existed before the invention of windmills.

Third factor C (the common-causal variable) causes both A and B [edit]

Main article: Spurious relationship

The **third-cause fallacy** (also known as *ignoring a common cause*[9] or *questionable cause*[9]) is a logical fallacy in which a spurious relationship is confused for causation. It asserts that X causes Y when in reality, both X and Y are caused by Z. It is a variation on the *post hoc ergo propter hoc* fallacy and a member of the questionable cause group of fallacies.

All of those examples deal with a lurking variable, which is simply a hidden third variable that affects both causes of the correlation. A difficulty often also arises where the third factor, though fundamentally different from A and B, is so closely related to A and/or B as to be confused with them or very difficult to scientifically disentangle from them (see Example 4).

Example 1

Sleeping with one's shoes on is strongly correlated with waking up with a headache.

Therefore, sleeping with one's shoes on causes headache.

The above example commits the correlation-implies-causation fallacy, as it prematurely concludes that sleeping with one's shoes on causes headache. A more plausible explanation is that both are caused by a third factor, in this case going to bed drunk, which thereby gives rise to a correlation. So the conclusion is false.

Bidirectional causation: A causes B, and B causes A [edit]

Causality is not necessarily one-way;[*dubious – discuss*] in a predator-prey relationship, predator numbers affect prey numbers, but prey numbers, i.e. food supply, also affect predator numbers. Another well-known example is that cyclists have a lower Body Mass Index than people who do not cycle. This is often explained by assuming that cycling increases physical activity levels and therefore decreases BMI. Because results from prospective studies on people who increase their bicycle use show a smaller effect on BMI than cross-sectional studies, there may be some reverse causality as well. For example, people with a lower BMI may be more likely to want to cycle in the first place. [18]

Confidence Interval

refers to the probability that a population parameter will fall between a set of values for a certain proportion of times.

$$CI = \bar{x} \pm z\frac{s}{\sqrt{n}}$$

Given a confidence level γ (95% and 99% are typical values),
a CI is a random interval which contains the parameter being estimated γ % of the time.

CI	= confidence interval
$\bar{x}$	= sample mean
z	= confidence level value
s	= sample standard deviation
n	= sample size

Say we have a rv X that represents heights,
following some distribution. (not necessarily Gaussian)
we have 10 observations. x1, x2, ..., x10.
if we want to estimate population mean mu,
we could assume

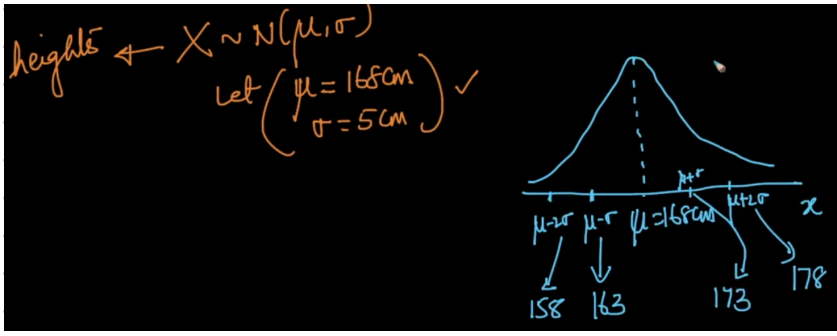
$$\mu = \bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

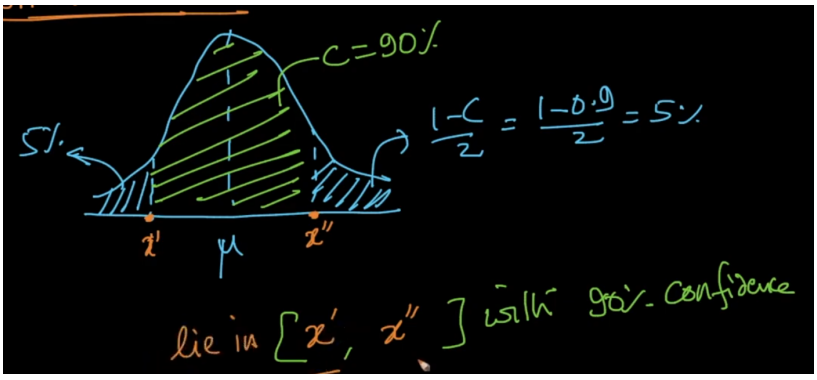
it has problems. as n increases,
\bar{x} moves closer to \mu
but they aren't exactly equal.
this is a point estimate. instead we could make an interval estimate.
it is good, but there's a better way

Consider the Gaussian distribution. $X \sim \mathcal{N}(\mu, \sigma)$

for a gaussian distribution,
95% data lies betn 2 std deviations of the mean

so we could say that heights of people lie
in [158, 178] with 95% confidence





we use normal distribution tables.
to find values of x' and x''

for 95%, it is $[\mu - 2\sigma, \mu + 2\sigma]$

C.I for mean(μ) of a r.v

$X \sim F$ with pop-mean of μ & std-dev of σ

$\downarrow \{x_1, x_2, \ldots, x_{10}\} \rightarrow$ Sample of size $= n = 10$

$\checkmark \{180, 162, 155, 172, 168, 150, 171, 183, 165, 176\}$

(Q) What is the 95% C.I of μ

2 cases:
we know population standard deviation
or
we don't

Case 1: $\sigma = 5$cm {we know pop-std-dev}

CLT: $\bar{x}$ = Sample mean $= \frac{1}{10}\sum_{i=1}^{10} x_i$ $(n=10)$

$\bar{x} \sim N\left(\mu, \frac{\sigma}{\sqrt{n}}\right)$ → CLT

Gaussian

$\bar{x}$: Sample-mean; μ: pop-mean; $\frac{\text{pop-std-dev}}{\sqrt{n}}$

$\mu \in \left[\bar{x} - \frac{2\sigma}{\sqrt{n}}, \bar{x} + \frac{2\sigma}{\sqrt{n}}\right]$ with 95% Confidence

$\mu \in [165.34, 171.66]$ with 95% Confidence

Case 2: if we dont know σ (pop-std-dev)

Sample of n

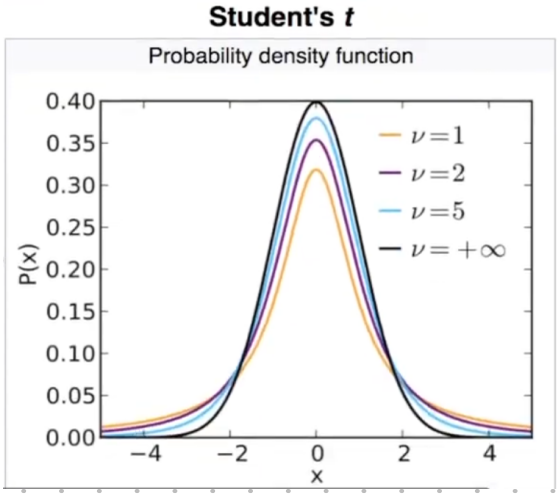
t-dist — Student's t-dist

$\bar{x} \sim t(n-1)$

$\bar{x}$: Sample mean; t: t-dist; $n-1$: degrees of freedom

Student's t distribution

the whole point of t distributions is to compute confidence intervals when sample size is small and population standard deviation is unknown



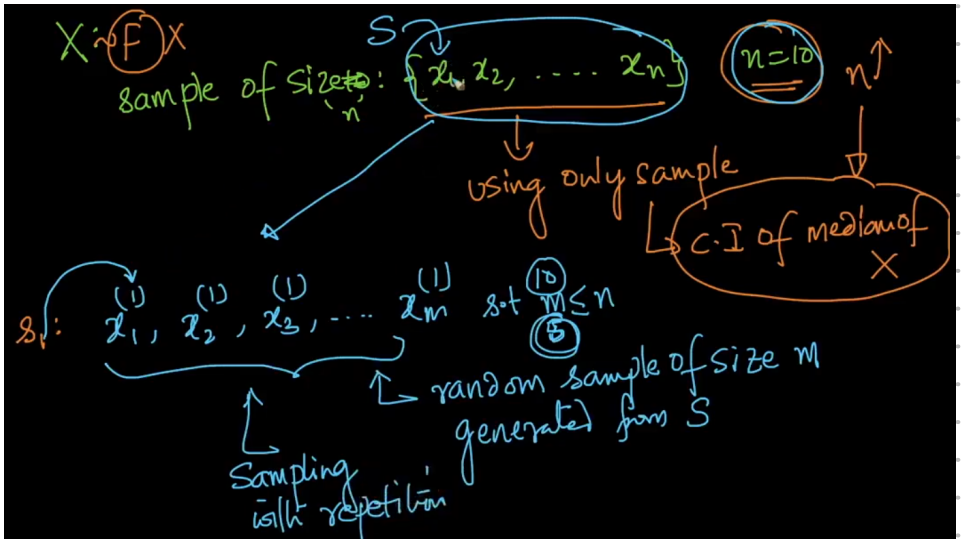
degree of freedom reflects peaked-ness

how do we compute confidence intervals for other statistics like median, std deviation, etc?

we use bootstrap based confidence intervals

CI using empirical bootstrap (non-parametric technique)

doesn't make assumption about distribution



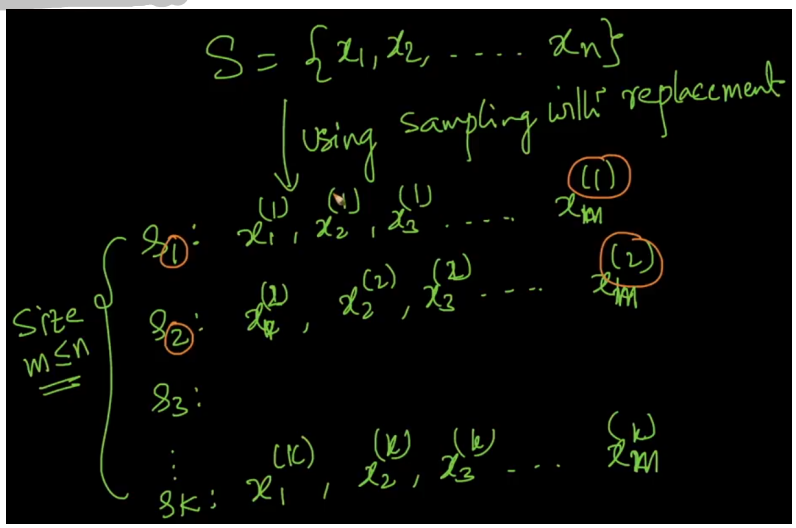
we generate a random sample s1 (of size m) from the actual sample S (size n) using uniform distribution U(1, n=10)
we get 10 values by using random function 10 times
say we get 5 first time, then x5 in S would be the first x1(1) in s1

using sampling with replacement, we create new samples
s1, s2, s3, ..., sk called bootstrap samples

we get all of those samples using discrete uniform distribution

say k=1000

we compute medians
m1, m2, ..., mk
for those bootstrap samples
s1, s2, ..., sk



sort them $m'_1 \leq m'_2 \leq \ldots \leq m'_{1000}$

say we want CI at 95% confidence

95% of 1000 is 950
5% is 50.
25 on the either side

95% CI is $[m'_{25}, m'_{975}]$

Empirical bootstrap based Confidence Interval

```
import numpy
from pandas import read_csv
from sklearn.utils import resample
from sklearn.metrics import accuracy_score
from matplotlib import pyplot

# load dataset
x = numpy.array([180,162,158,172,168,150,171,183,165,176])

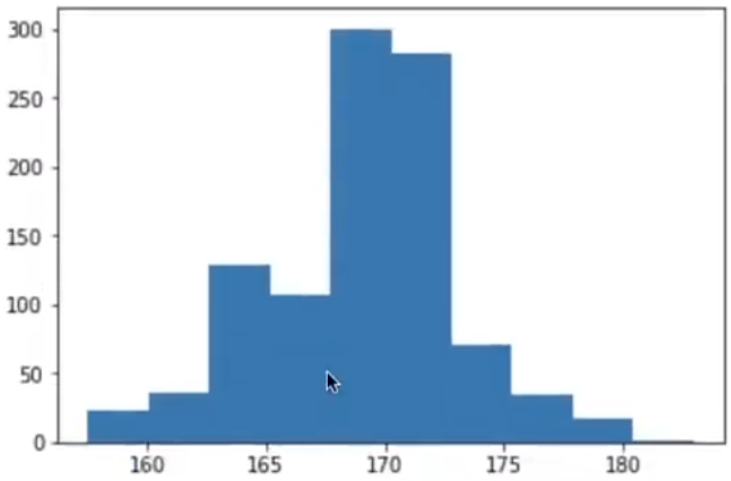
# configure bootstrap
n_iterations = 1000
n_size = int(len(x))
```

```
# run bootstrap
medians = list()
for i in range(n_iterations):
    # prepare train and test sets
    s = resample(x, n_samples=n_size);
    m = numpy.median(s);
    #print(m)
    medians.append(m)
```

```
# plot scores
pyplot.hist(medians)
pyplot.show()

# confidence intervals
alpha = 0.95
p = ((1.0-alpha)/2.0) * 100
lower =  numpy.percentile(medians, p)

p = (alpha+((1.0-alpha)/2.0)) * 100
upper =  numpy.percentile(medians, p)
print('%.1f confidence interval %.1f and %.1f' % (alpha*100, lower, upper))
```



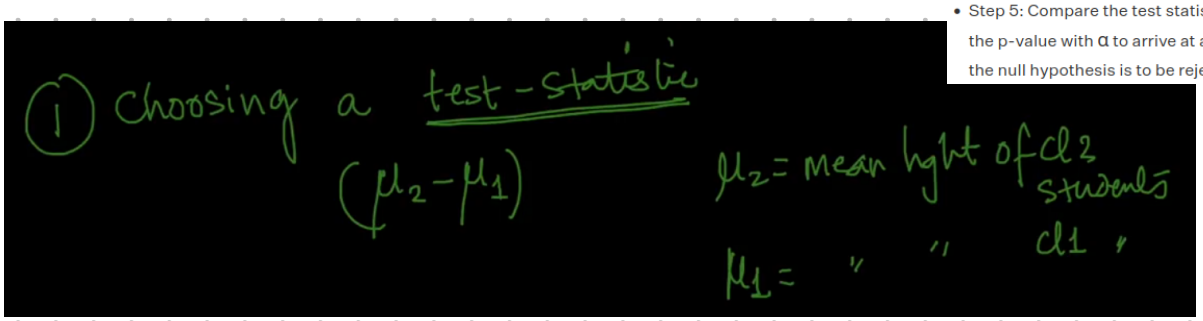

```
95.0 confidence interval 161.5 and 176.0
```

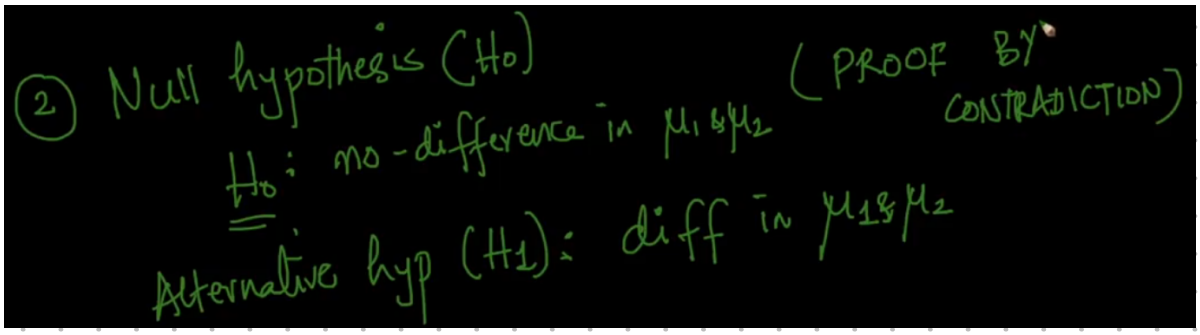
Hypothesis Testing

The typical high level steps involved in performing a frequentist hypothesis test in practice are:

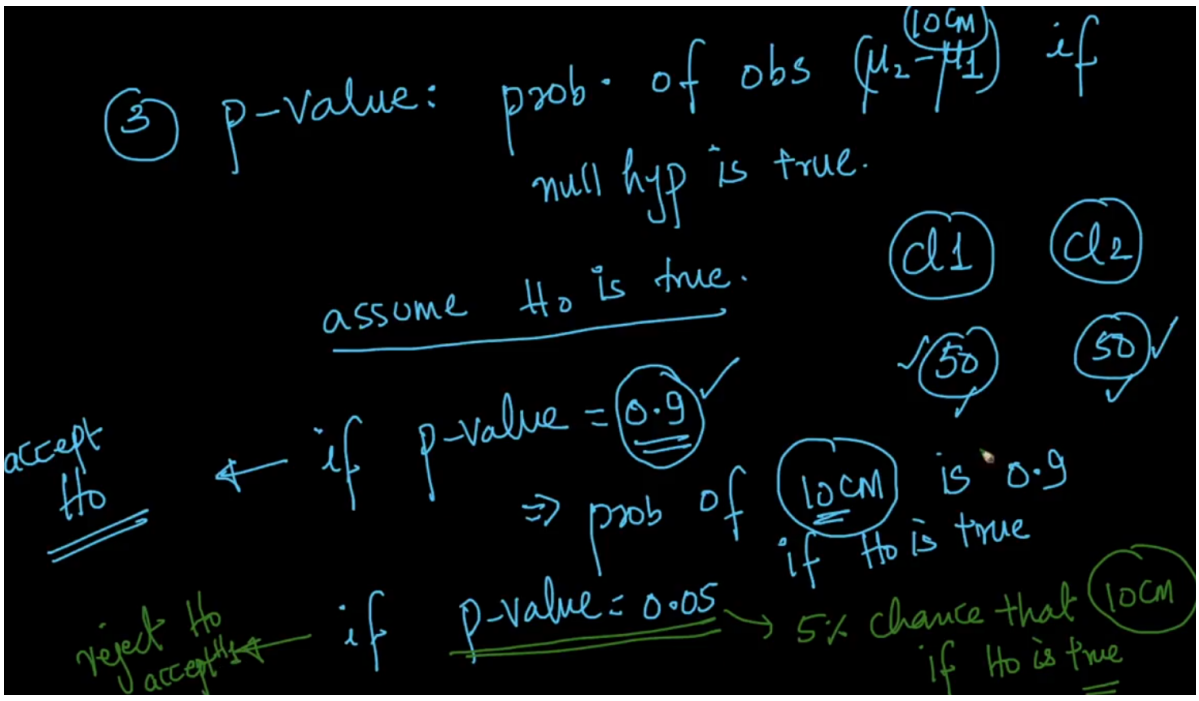
1. Define a hypothesis (claim which is testable using data).
2. Select a relevant statistical test with associated test statistic *T*.
3. Derive the distribution of the test statistic under the null hypothesis from the assumptions. In standard cases this will be a well-known result. For example, the test statistic might follow a Student's t distribution with known degrees of freedom, or a normal distribution with known mean and variance.
4. Select a significance level (α), the maximum acceptable false positive rate. Common values are 5% and 1%.
5. Compute from the observations the observed value t_{obs} of the test statistic *T*.
6. Decide to either reject the null hypothesis in favor of the alternative or not reject it. The Neyman-Pearson decision rule is to reject the null hypothesis H_0 if the observed value t_{obs} is in the critical region, and not to reject the null hypothesis otherwise.[2][3]

- Step 1: Set up the null hypothesis by correctly identifying whether it is the left-tailed, right-tailed, or two-tailed hypothesis testing.
- Step 2: Set up the alternative hypothesis.
- Step 3: Choose the correct significance level, α, and find the critical value.
- Step 4: Calculate the correct test statistic (z, t or χ) and p-value.
- Step 5: Compare the test statistic with the critical value or compare the p-value with α to arrive at a conclusion. In other words, decide if the null hypothesis is to be rejected or not.





we assume H_0 is true, then try to prove otherwise.
we accordingly accept or reject H_0



high p value would mean a higher chance that H_0 is true

low p value would mean a lesser chance that H_0 is true

example 1: Given a coin, determine if it is biased towards heads

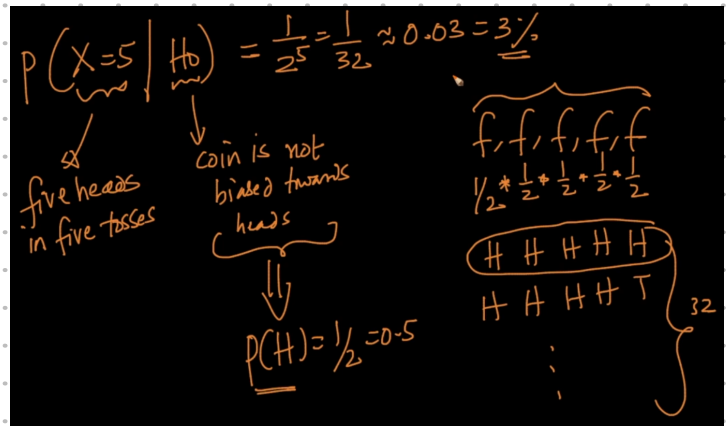
biased towards heads means P(H) > 0.5

X: # of heads -- test statistic (on flipping n times, n=5)

say our expt results HHHHH, X=5

we assume
H_0: coin is not biased towards heads

we calculate the P(observation | H_0)
i.e. P(X=5|coin is not biased towards heads)



there's a 3% chance of
getting 5 heads in 5 tosses
if the coin is not biased towards heads

P(observation | H_0) = 3%, quite low

so here p-value = 0.03

at alpha = 0.05 (5%)
p-value < 0.05

so we reject H_0, the idea that coin is not biased towards heads

so we'll say H_0 must be incorrect

so we accept that coin is biased towards heads

results of the experiment may change when the # of trials is changed (sample size)

so design of expt is imp.

P-value: Permutation Testing

we have 2 classes cl1, cl2 each with 50 samples. $\Delta = \mu_{cl1} - \mu_{cl2}$
difference in means μ1-μ2 = Δ

we jumble them. then randomly sample 50.
50 go in X, remaining 50 go in Y
we compute means μ1, μ2
we get μ1-μ2 = δ1

we do all of it again, get μ1-μ2 = δ2
we do it 10,000 times, we get

δ1, δ2, ..., δ10k
we sort them

we need to check where Δ fits in sorted order
δ1', δ2', ..., δ10k'

δ1', δ2', ..., Δ, ..., δ10k'

say there are 5% values in sorted order > Δ
then p-value = 0.05

what's the point of all of this?

we assume that there's no difference between the classes
i.e. we assume H_0

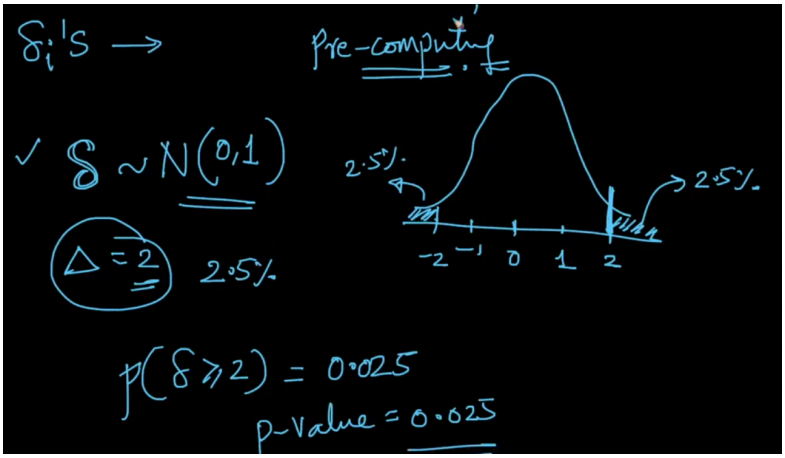
if there's no difference betn heights of students of classes 1 & 2
if we resample, we ought to get similar classes on random sampling

δ's could vary

if Δ > δ 95% of the times, then we can't say that there's no difference
if p-value = 0.20, ie Δ > 80% of δ's then if we assume no difference, we are observing there's a 20% chance that we'll get greater difference, so it's not something too absurd to get that Δ.
so we'll accept H_0

it wasn't always feasible to perform those 10,000 resampling and all so it was assumped that deltas come from a distribution

e.g. if delta comes from N(0, 1)
we could say values > 2 would have a chance of 2.5%



Kolmogorov–Smirnov test (non-parametric)

two sample K-S test:
we have X1 (n samples), X2 (m samples)
we want to check if both have same distribution

K-S statistic is

$$D_{n,m} = \sup_x |F_{1,n}(x) - F_{2,m}(x)|$$

where F_{1,n} and F_{2,m} are the empirical distribution functions
of the first and the second sample respectively,
and sup is the supremum function.

For large samples, the null hypothesis is rejected at level α if

$$D_{n,m} > c(\alpha)\sqrt{\frac{n+m}{n \cdot m}} \qquad \text{where} \qquad c(\alpha) = \sqrt{-\ln\left(\frac{\alpha}{2}\right) \cdot \frac{1}{2}}$$

essentially, we take difference betn
CDF's of 2 variables
if we have very large n & m,
CDF's of x1 and x2 would quite overlap
if they were from the same distribution

but for smaller n & m, there's a difference

supremum function is the maximal difference between the CDF's

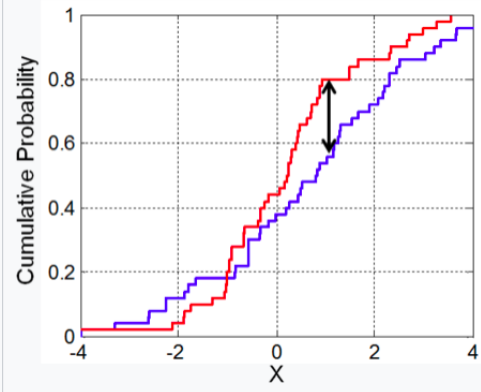


Illustration of the two-sample Kolmogorov–Smirnov statistic. Red and blue lines each correspond to an empirical distribution function, and the black arrow is the two-sample KS statistic.

scipy.stats.kstest(*rvs*, *cdf*, *args=()*, *N=20*, *alternative='two-sided'*, *method='auto'*)

Performs the (one-sample or two-sample) Kolmogorov-Smirnov test for goodness of fit.

The one-sample test compares the underlying distribution F(x) of a sample against a given distribution G(x). The two-sample test compares the underlying distributions of two independent samples. Both tests are valid only for continuous distributions.

Parameters:

rvs : *str, array_like, or callable*

If an array, it should be a 1-D array of observations of random variables. If a callable, it should be a function to generate random variables; it is required to have a keyword argument *size*. If a string, it should be the name of a distribution in `scipy.stats`, which will be used to generate random variables.

cdf : *str, array_like or callable*

If array_like, it should be a 1-D array of observations of random variables, and the two-sample test is performed (and rvs must be array_like). If a callable, that callable is used to calculate the cdf. If a string, it should be the name of a distribution in `scipy.stats`, which will be used as the cdf function.

args : *tuple, sequence, optional*

Distribution parameters, used if *rvs* or *cdf* are strings or callables.

N : *int, optional*

Sample size if *rvs* is string or callable. Default is 20.

alternative : *{'two-sided', 'less', 'greater'}, optional*

Defines the null and alternative hypotheses. Default is 'two-sided'. Please see explanations in the Notes below.

method : *{'auto', 'exact', 'approx', 'asymp'}, optional*

Defines the distribution used for calculating the p-value. The following options are available (default is 'auto'):

- 'auto' : selects one of the other options.
- 'exact' : uses the exact distribution of test statistic.
- 'approx' : approximates the two-sided probability with twice the one-sided probability
- 'asymp': uses asymptotic distribution of test statistic

Returns:

res: KstestResult

An object containing attributes:

statistic : *float*

KS test statistic, either D+, D-, or D (the maximum of the two)

pvalue : *float*

One-tailed or two-tailed p-value.

statistic_location : *float*

In a one-sample test, this is the value of *rvs* corresponding with the KS statistic; i.e., the distance between the empirical distribution function and the hypothesized cumulative distribution function is measured at this observation.

In a two-sample test, this is the value from *rvs* or *cdf* corresponding with the KS statistic; i.e., the distance between the empirical distribution functions is measured at this observation.

statistic_sign : *int*

In a one-sample test, this is +1 if the KS statistic is the maximum positive difference between the empirical distribution function and the hypothesized cumulative distribution function (D+); it is -1 if the KS statistic is the maximum negative difference (D-).

In a two-sample test, this is +1 if the empirical distribution function of *rvs* exceeds the empirical distribution function of *cdf* at *statistic_location*, otherwise -1.

Examples

Suppose we wish to test the null hypothesis that a sample is distributed according to the standard normal. We choose a confidence level of 95%; that is, we will reject the null hypothesis in favor of the alternative if the p-value is less than 0.05.

When testing uniformly distributed data, we would expect the null hypothesis to be rejected.

```
>>> import numpy as np
>>> from scipy import stats
>>> rng = np.random.default_rng()
>>> stats.kstest(stats.uniform.rvs(size=100, random_state=rng),
...              stats.norm.cdf)
KstestResult(statistic=0.5001899973268688, pvalue=1.1616392184763533e-23)
```

Indeed, the p-value is lower than our threshold of 0.05, so we reject the null hypothesis in favor of the default "two-sided" alternative: the data are *not* distributed according to the standard normal.

When testing random variates from the standard normal distribution, we expect the data to be consistent with the null hypothesis most of the time.

```
>>> x = stats.norm.rvs(size=100, random_state=rng)
>>> stats.kstest(x, stats.norm.cdf)
KstestResult(statistic=0.05345882212970396, pvalue=0.9227159037744717)
```

As expected, the p-value of 0.92 is not below our threshold of 0.05, so we cannot reject the null hypothesis.

Suppose, however, that the random variates are distributed according to a normal distribution that is shifted toward greater values. In this case, the cumulative density function (CDF) of the underlying distribution tends to be *less* than the CDF of the standard normal. Therefore, we would expect the null hypothesis to be rejected with `alternative='less'`:

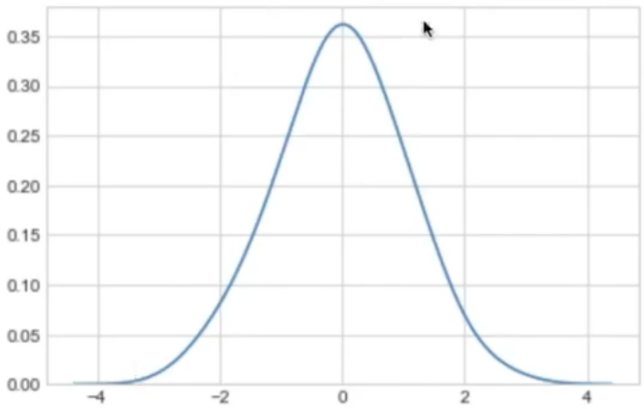
```
>>> x = stats.norm.rvs(size=100, loc=0.5, random_state=rng)
>>> stats.kstest(x, stats.norm.cdf, alternative='less')
KstestResult(statistic=0.17482387821055168, pvalue=0.001913921057766743)
```

and indeed, with p-value smaller than our threshold, we reject the null hypothesis in favor of the alternative.

K-S Test

```
import numpy as np
import seaborn as sns
from scipy import stats
import matplotlib.pyplot as plt

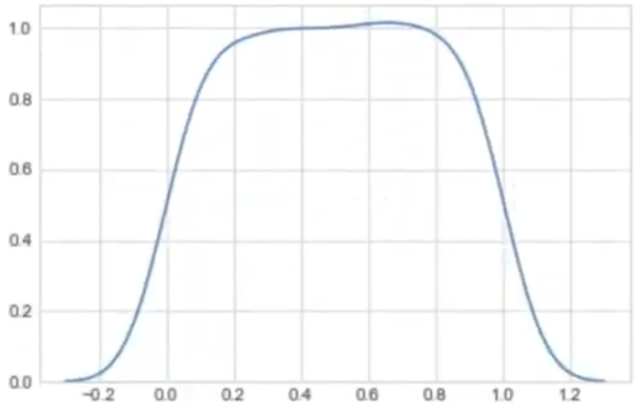
#generate a gaussian r.v X
x = stats.norm.rvs(size=1000);
sns.set_style('whitegrid')
sns.kdeplot(np.array(x), bw=0.5)
plt.show()
```




```
stats.kstest(x, 'norm')
```

```
KstestResult(statistic=0.021308397286061931, pvalue=0.75424031453335627)
```

```
# Y ~ Continous Uniform Distribution(0,1)
y = np.random.uniform(0,1,10000);
sns.kdeplot(np.array(y), bw=0.1)
plt.show()
```




```
stats.kstest(y, 'norm')
```

```
KstestResult(statistic=0.50159644397739489, pvalue=0.0)
```

example 2: difference of means

we have 2 citities c1 and c2. we want to determine if population means of heights of people of both is the same

we obv can't calc population mean, so we use sample mean.
we randomly sample 50 people from both.

C1: { h1, h2, ..., h50 } sample mean = \mu_1
C2: {h1', h2', ..., h50'} sample mean = \mu_2

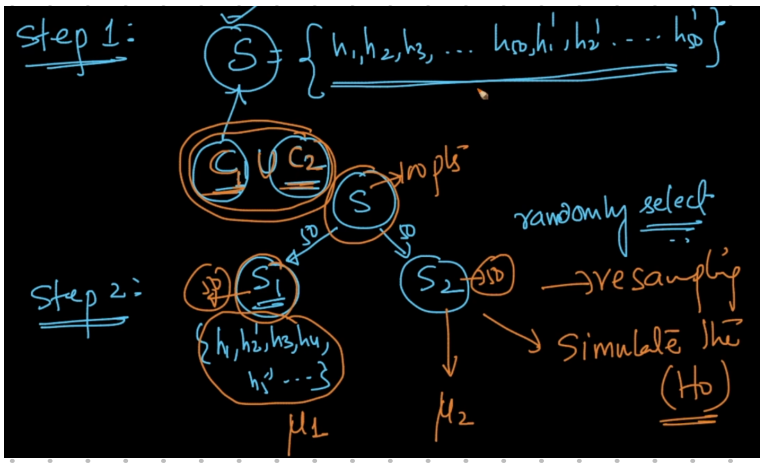
say \mu_1 = 162cm, \mu_2 = 167cm

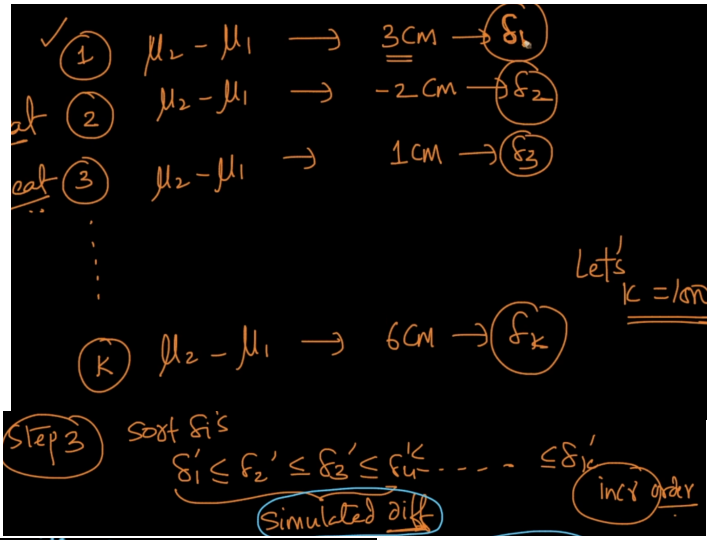
let the test statistic be $\mu_2 - \mu_1$ = 167-162 = 5cm

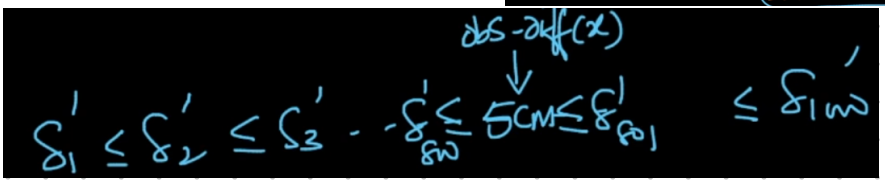
$H_0 : \mu_1 = \mu_2$ H_0: there's no difference between the means

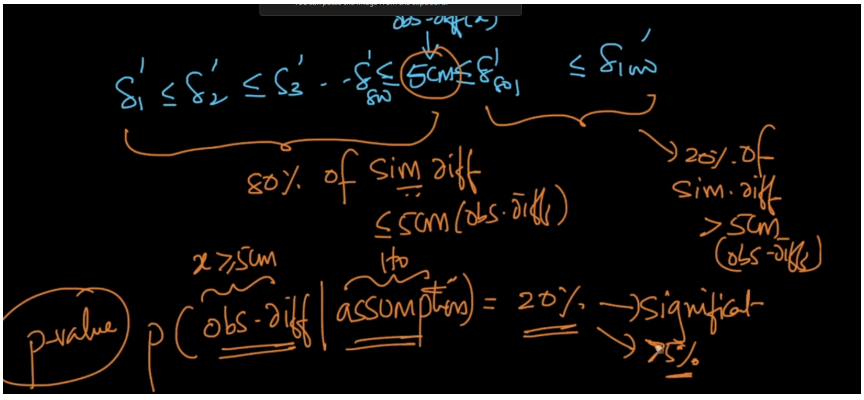
we'll compute
P(X=5cm | H_0) here X is the difference betn means
= P(observing 5cm diff in sample means | there's no diff in pop means)

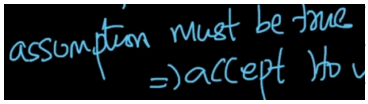
to compute p value, use resampling and permutation test





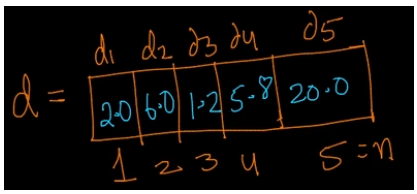






Proportional Sampling

say we have a 1xn matrix
the task is to pick an element amongst these n elements s.t. the probability of picking an element is proportional to its value

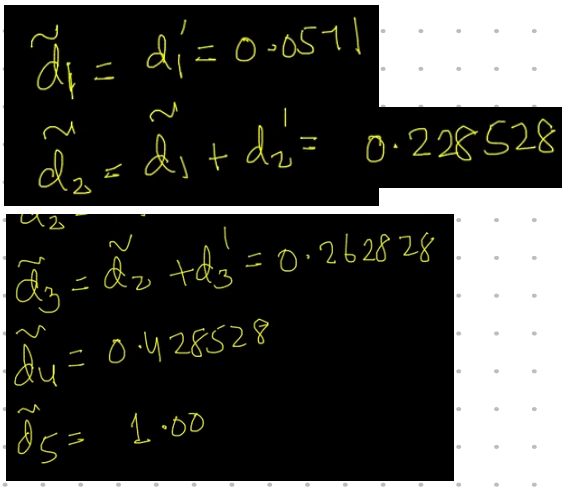


20 here is much larger than the others
so we want its probabability to be picked greater lly, 1.2 should be picked less

step 1.a. calculate sum of all di's, S = 35
1.b. compute di' = di/S ex. d1' = 20/35 = 0.571428
(basically normalizing values using sum)

new values are 0.0571, 0.171428, 0.0343, 0.1657, 05714
now these values lie betn and sum to 1

step 1.c. compute the cumulative normalized sum $d_i^{\sim} = d'_i + d'_{i-1} + ... + d'_1$



step 2. sample 1 value r from uniform distribution U(0.0, 1.0)

```
r = np.random.uniform(0.0, 1.0, 1)
```

let r = 0.6

step 3. use d_i' values

